

# Q3A: Sonic Signatures

## Magical Mystery Tune

EE200: Signals, Systems and Networks — Course Project, Summer 2026

Neelam Meena

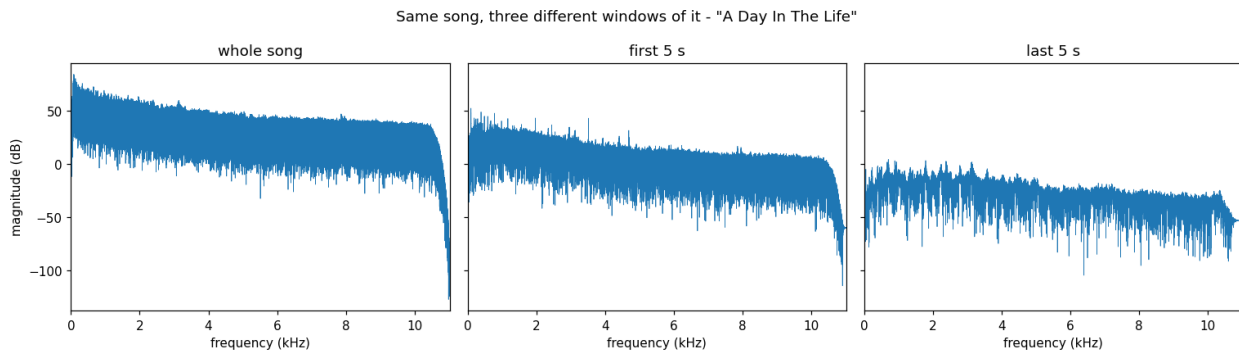
## 1 Introduction

A music identification app such as Shazam never compares raw audio waveforms. Waveform comparison would be both far too slow and far too sensitive to the noise, compression artefacts, and pitch wobble that any real recording picks up. Instead, the audio is converted into a sparse fingerprint — a small set of distinctive time-frequency points — and a query clip is identified by finding which song’s fingerprint lines up with the query’s fingerprint at a single, consistent time offset.

This report builds and tests a small version of that pipeline: the limitation of a single Fourier transform, the spectrogram, peak extraction (the “constellation”), pairwise hashing, database construction, offset-histogram matching, and three robustness experiments (noise, pitch shift, time stretch). All numbers and figures below come from running the accompanying notebook on the actual 50-song library provided for the course ([EE200 Project Song Database](#)); none of it is simulated or invented.

## 2 Why a Single Fourier Transform Is Not Enough

The DFT of an entire song collapses its whole time history into one spectrum. We learn *which* frequencies were present somewhere in the recording, but every clue about *when* they occurred is gone. To make this concrete, we took one song (*A Day In The Life*, 337.4s) and computed three full-length-equivalent magnitude spectra: of the whole song, of just its first five seconds, and of just its last five seconds.



**Figure 1:** DFT magnitude of the whole song versus its first and last five seconds. All three traces show essentially the same broad frequency content, even though the start and end of the song are musically very different sections.

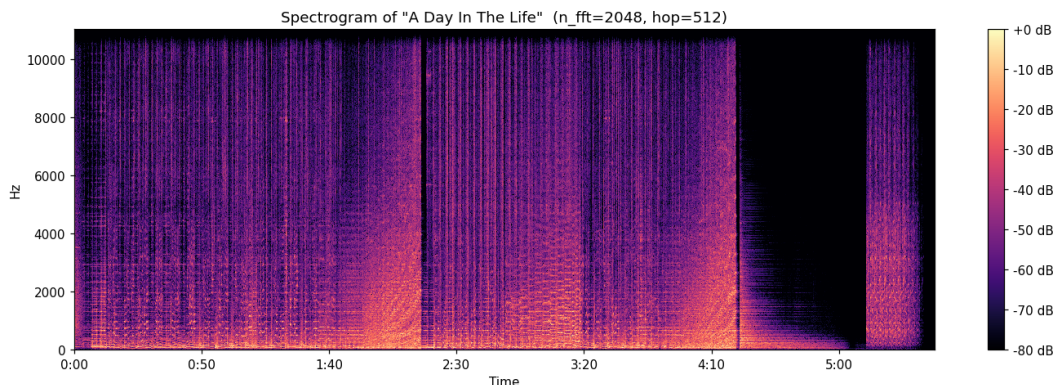
The three spectra are nearly indistinguishable in their broad shape. A single DFT cannot tell which part of a song is playing, let alone identify the song against a library of others — it has thrown away the one axis (time) that recognition actually depends on.

### 3 The Spectrogram

The fix is the short-time Fourier transform (STFT): slide a window of length  $N$  across the signal with hop  $H$ , and take a DFT at every position:

$$X[k, m] = \sum_{n=0}^{N-1} x[n + mH] w[n] e^{-j2\pi kn/N}. \quad (1)$$

Stacking  $|X[k, m]|$  (in dB) as columns gives a 2-D picture with time on the horizontal axis and frequency on the vertical axis — the spectrogram. We used  $N=2048$  samples with a hop of 512 samples at  $f_s = 22050$  Hz, giving a time resolution of about 23.2 ms per frame and a frequency resolution of about 10.8 Hz per bin.

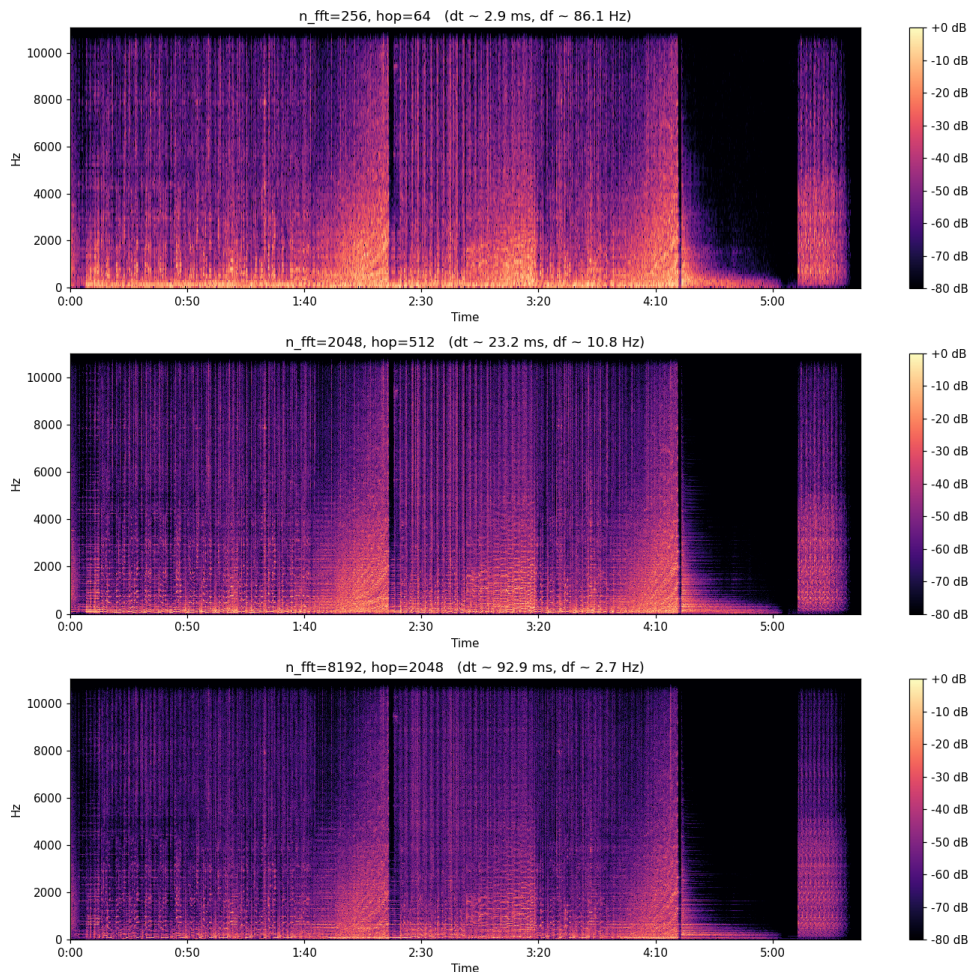


**Figure 2:** Spectrogram of “A Day In The Life” ( $n_{fft}=2048$ ,  $hop=512$ ). The song’s distinct sections (the verse/orchestral build, the famous orchestral crescendo around 4:10, and the closing piano chord) are all clearly visible along the time axis.

#### 3.1 Window-length trade-off

The STFT has a fundamental trade-off, related to the time-frequency uncertainty bound  $\Delta t \cdot \Delta f \gtrsim 1/(4\pi)$ : a short window localises events precisely in time but blurs frequency, and a long window does the opposite. We recomputed the same spectrogram with  $n_{fft} = 256$  (short) and  $n_{fft} = 8192$  (long), keeping the same song.

With  $n_{fft} = 256$ , transient events (the orchestral hits, drum accents) are sharply localised in time, but the harmonic content smears into thick horizontal bands, so individual notes are hard to separate. With  $n_{fft} = 8192$ , the opposite happens: harmonic lines become thin and sharp and close pitches are easy to tell apart, but anything percussive smears across nearly 100 ms, losing its precise timing.  $n_{fft} = 2048$  sits in between, and is what we use for the rest of the pipeline, because fingerprinting needs both a fairly precise time location for each peak and enough frequency separation to make peaks distinctive.



**Figure 3:** Same song at three window lengths. Top:  $n_{fft} = 256$  ( $\Delta t \approx 2.9$  ms,  $\Delta f \approx 86.1$  Hz). Middle:  $n_{fft} = 2048$  ( $\Delta t \approx 23.2$  ms,  $\Delta f \approx 10.8$  Hz). Bottom:  $n_{fft} = 8192$  ( $\Delta t \approx 92.9$  ms,  $\Delta f \approx 2.7$  Hz).

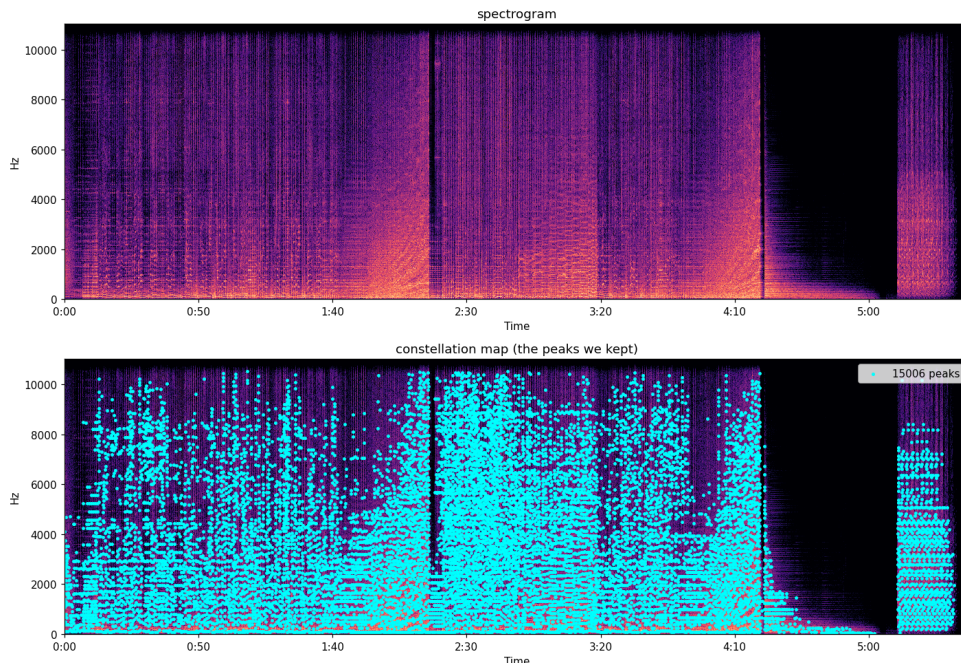
## 4 Constellation Map

A full spectrogram has millions of cells per song, far too much to use directly as a fingerprint. We keep only the local maxima of the spectrogram in dB — points that are louder than every neighbour in a  $20 \times 20$  time-frequency window and within 40 dB of the song’s loudest point. These peaks are chosen specifically because they are robust: a loud, locally dominant point tends to remain the dominant point even with some noise added, whereas an arbitrary low-energy sample does not.

## 5 Hashing and the Song Database

A single peak is just one frequency bin, which is far too common on its own to identify a song (every song has *some* energy near any given frequency). Following the standard “Shazam-style” approach, we instead pair each peak (the anchor) with up to 15 peaks that follow it within 200 frames, and hash the pair:

$$h = (f_1, f_2, \Delta t).$$



**Figure 4:** Top: spectrogram of “A Day In The Life”. Bottom: the same spectrogram with its 15,006 constellation peaks overlaid in cyan — a  $992\times$  reduction from the 14,896,325 raw spectrogram cells.

Two specific frequencies separated by a specific number of frames is a far rarer coincidence than either frequency alone, which is exactly what makes pair hashes so much more selective than single-peak hashes (Section 7).

For “A Day In The Life” alone, the 15,006 peaks produced 224,643 pair hashes (208,210 of them unique) versus only 15,006 single-peak hashes (spread over just 968 unique keys — the entire usable frequency axis). Running this over the full 50-song library:

| Quantity                      | Value     |
|-------------------------------|-----------|
| Songs indexed                 | 50        |
| Total unique pair-hash keys   | 5,860,314 |
| Total unique single-hash keys | 994       |

**Table 1:** Database statistics over the full provided song library.

The single-hash key space stays essentially fixed at under 1,000 keys no matter how many songs are added, while the pair-hash key space grows into the millions — this gap is the entire reason pairing works.

## 6 Identifying a Query Clip

To identify an unknown clip, we fingerprint it exactly the same way and look up every one of its hashes in the database. Each hit gives an offset  $\Delta = t_{db} - t_q$  between the database anchor time and the query anchor time. If the clip really is from that song, essentially every matching hash points to the *same* offset, since the whole clip is a constant time-shift of the original track; for an unrelated

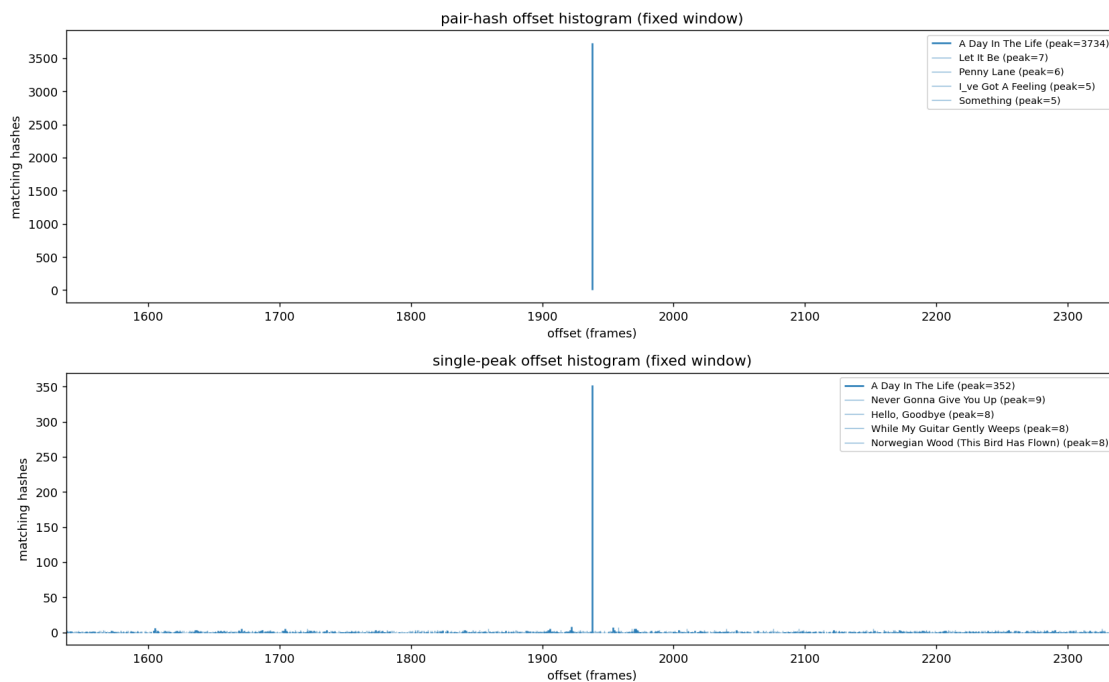


song, matches are coincidental and their offsets scatter randomly. The song whose offset histogram has the tallest single bin is the identification.

We tested this with a clean 10-second clip cut from 0:45–0:55 of “A Day In The Life” (a part of the song not used to build the model song’s identity in any special way — just an arbitrary 10-second window).

| Scheme      | Identified song             | Winning offset-bin score |
|-------------|-----------------------------|--------------------------|
| Pair hash   | A Day In The Life (correct) | 3724                     |
| Single peak | A Day In The Life (correct) | 354                      |

**Table 2:** Clean-clip identification result. Both schemes get the right answer; the margin over wrong songs is what differs (Section 7).

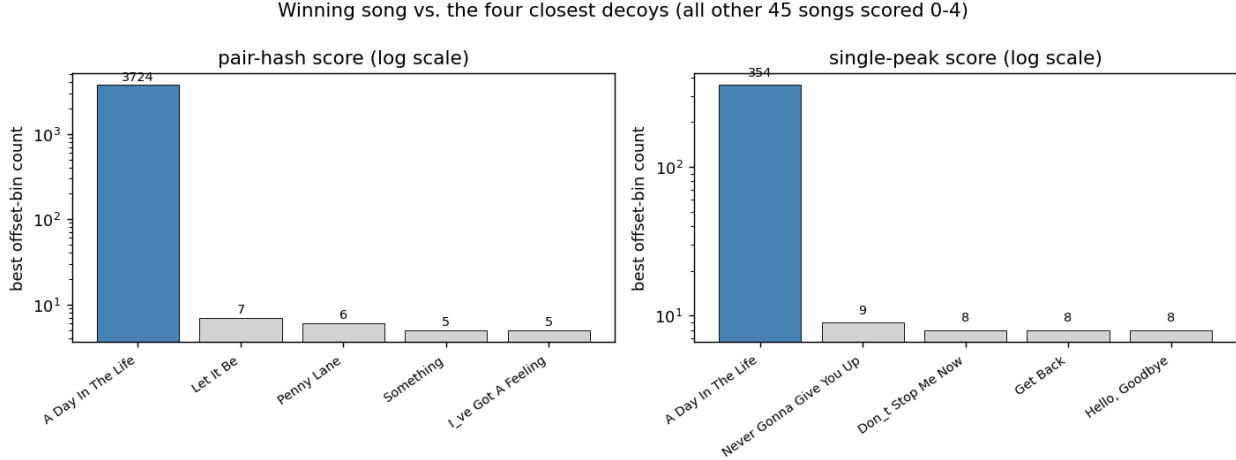


**Figure 5:** Offset histograms for the same clip, zoomed in around the winning offset (frame  $\approx 1937$ , i.e.  $\approx 45$ s into the song, matching exactly where the clip was cut from). *Top:* the pair-hash histogram shows one extremely tall, completely isolated spike; every decoy song in this window scores 5–7. *Bottom:* the single-peak histogram shows the same correct spike, but sitting on top of a visibly noisy floor of small chance matches scattered across the whole offset range.

**A note on a plotting pitfall.** Our first version of this histogram plot used a fixed  $x$ -axis window of  $[-600, 600]$  frames around zero. Because this query clip was cut from 45s into the song (frame offset  $\approx 1937$ ), the true matching spike fell completely outside that fixed window, and the panel appeared blank even though the underlying score (3724) was being computed correctly the whole time — the legend showed the right peak value, but the line itself was off-screen. Figure 5 uses a window that is centred on the winning song’s *actual* best offset instead of a hardcoded range, which is the correct general fix: the right offset depends on where in the song a query clip happens to start, not on any fixed constant.

## 7 Single Peaks vs. Paired Hashes

Why does pairing two peaks together make identification so much more decisive? It comes down to the probability of an unrelated, coincidental match. A single hash is one frequency bin out of roughly  $B \approx 1000$  usable bins, so two unrelated songs will share individual frequency bins constantly just by chance — which is exactly why the single-peak histogram in Figure 5 has a visibly noisy floor. A pair hash is a triple drawn from a space of order  $B^2 \cdot T$  (with  $T \approx 200$  the  $\Delta t$  window), several orders of magnitude larger, so two unrelated songs sharing the same frequency pair at the same time gap is a far rarer coincidence.



**Figure 6:** Best offset-bin score for the winning song versus its four closest decoys, log scale (all other 45 songs in the library scored between 0 and 4). The pair hash wins by a factor of about  $530\times$  over the nearest decoy; the single-peak hash wins by only about  $39\times$ .

On a clean, undistorted clip, both schemes happened to identify the song correctly — but the *margin* of confidence is where pair hashing earns its keep, and that margin is exactly what determines how much noise or distortion the system can tolerate before a wrong song’s coincidental score catches up to the right one.

## 8 Robustness

We re-used the same 10-second clip from Section 6 for every test below, so all three experiments are directly comparable.

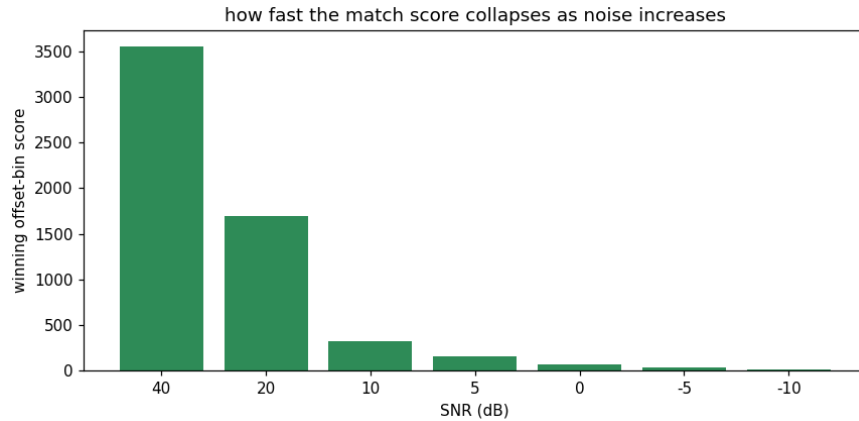
### 8.1 Noise

We added white Gaussian noise at decreasing SNR and re-ran identification with the pair-hash scheme.

Each surviving peak that still matches votes for the same correct offset, so as long as *some* real peaks survive the noise, the genuine spike keeps standing out, even as it gets shorter and shorter. The score is falling by roughly an order of magnitude every 10–15 dB, so a slightly noisier or shorter query clip, or a song with fewer/weaker constellation peaks to begin with, would push the breaking point to a noticeably higher SNR than what we observed here — the system is robust, not invincible.

| SNR (dB) | Winning score | Correct? |
|----------|---------------|----------|
| 40       | 3549          | yes      |
| 20       | 1690          | yes      |
| 10       | 327           | yes      |
| 5        | 159           | yes      |
| 0        | 68            | yes      |
| -5       | 30            | yes      |
| -10      | 8             | yes      |

**Table 3:** Identification under additive white Gaussian noise.



**Figure 7:** Winning offset-bin score versus SNR. All bars are green because the identification stayed correct all the way down to  $-10$  dB in this test — the score shrinks dramatically ( $3549 \rightarrow 8$ ) but never quite lets a wrong song’s coincidental score overtake it.

## 8.2 Pitch shift

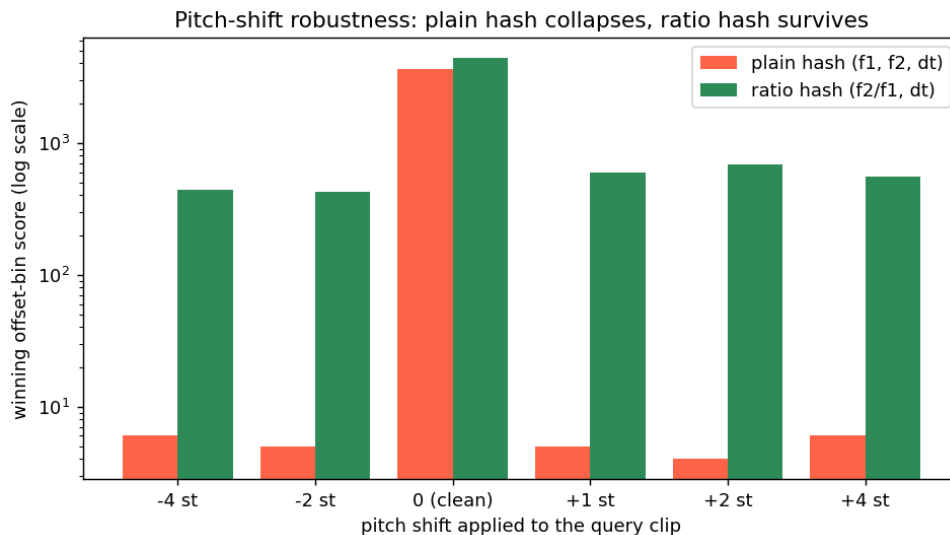
A pitch shift of  $n$  semitones multiplies every frequency by  $\alpha = 2^{n/12}$ . A hash that used to be  $(f_1, f_2, \Delta t)$  becomes  $(\alpha f_1, \alpha f_2, \Delta t)$ , and because the hash key is the *exact integer* frequency bin, even a one-semitone shift ( $\alpha \approx 1.059$ ) moves  $\alpha f_1$  into a different bin almost everywhere, so the key vanishes from the database.

This is the most counter-intuitive result in the whole project: at  $+1$  semitone the song sounds essentially identical to a human listener, yet the plain hash’s score collapses from 3645 to 5 — a complete failure, not a gentle degradation. None of the musical information changed; only the *integer bin index* that the hash table indexes on changed, and that was enough.

**Suggested fix, implemented and tested.** If  $f_1 \rightarrow \alpha f_1$  and  $f_2 \rightarrow \alpha f_2$  under a uniform pitch shift, the ratio  $f_2/f_1$  is unchanged, since  $\alpha$  cancels. Replacing the hash key with  $(\text{round}(f_2/f_1, 2), \Delta t)$  should be pitch-shift invariant by construction, and the table above confirms it: the ratio hash correctly identifies the song at every shift we tested, where the plain hash failed at all of them. The trade-off is a coarser key (rounding to two decimal places collides more often than an exact integer pair), which is why the ratio hash’s own clean-clip margin (4391) is not dramatically larger than the plain hash’s, and why its post-shift scores (420–680) are noticeably lower than its own unshifted score — it is more robust, not strictly better in every respect.

| Shift (semitones) | Plain-hash result                               | Ratio-hash result                  |
|-------------------|-------------------------------------------------|------------------------------------|
| 0                 | A Day In The Life (3645) – correct              | A Day In The Life (4391) – correct |
| +1                | Hey Jude (5) – <b>wrong</b>                     | A Day In The Life (599) – correct  |
| +2                | Hello, Goodbye (4) – <b>wrong</b>               | A Day In The Life (683) – correct  |
| +4                | While My Guitar Gently Weeps (6) – <b>wrong</b> | A Day In The Life (552) – correct  |
| −2                | In My Life (5) – <b>wrong</b>                   | A Day In The Life (427) – correct  |
| −4                | Norwegian Wood (6) – <b>wrong</b>               | A Day In The Life (443) – correct  |

**Table 4:** Pitch-shift robustness: the original  $(f_1, f_2, \Delta t)$  hash versus the pitch-invariant  $(\text{round}(f_2/f_1, 2), \Delta t)$  ratio hash.



**Figure 8:** Winning score versus pitch shift, log scale. The plain hash collapses by roughly 600–900× at even  $\pm 1$  semitone and never recovers; the ratio hash stays within a factor of 7–10 of its clean score across the whole range tested.

### 8.3 Time stretch

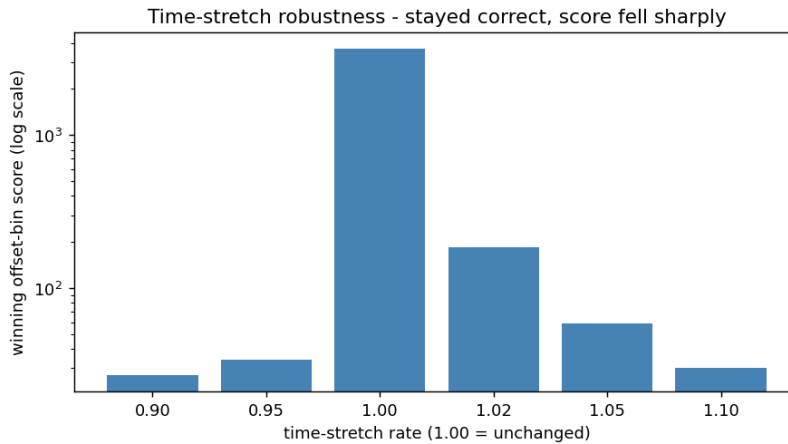
Stretching the clip in time changes how many frames separate any two peaks, so  $\Delta t$  drifts away from the value stored in the database, rather than  $f_1$  or  $f_2$ .

Time stretch turned out to be noticeably more forgiving than pitch shift in our tests: identification remained correct across every rate from 0.90 to 1.10. The reason is the  $\pm 1$ -frame rounding tolerance built into using integer frame indices for  $\Delta t$  — a modest stretch still rounds most pairs back to the same integer  $\Delta t$  they had before, so a fair number of hashes keep matching even though the score clearly degrades. A pitch shift gets no equivalent slack, because it acts multiplicatively on *every* frequency bin simultaneously rather than nudging a handful of frame indices, so essentially the entire hash population is invalidated at once rather than just the ones unlucky enough to round the wrong way.



| Rate | Winning score | Correct? |
|------|---------------|----------|
| 1.00 | 3645          | yes      |
| 1.02 | 184           | yes      |
| 1.05 | 59            | yes      |
| 1.10 | 30            | yes      |
| 0.95 | 34            | yes      |
| 0.90 | 27            | yes      |

**Table 5:** Identification under time-stretching (rate  $> 1$  = faster/shorter).



**Figure 9:** Winning score versus stretch rate, log scale. Identification stayed correct across the whole  $\pm 10\%$  range tested, but the score fell by roughly two orders of magnitude even at a 2% stretch.

## 9 Sanity Check Across the Whole Library

As a final check, we pulled one random 10-second clip from every one of the 50 songs (no added noise or distortion) and ran identification with both schemes.

Pair-hash accuracy: **50/50 (100%)**

Single-peak accuracy: **50/50 (100%)**

On clean audio, both schemes are perfectly accurate over the whole library — which is consistent with Section 7: the single-peak hash is not *inaccurate* on easy, undistorted input, it simply has a much smaller safety margin, and that margin is what disappears first under noise, pitch shift, or time stretch.

## 10 Conclusions

A single DFT of a whole song destroys all temporal structure and cannot identify music, because it cannot even tell two different sections of the *same* song apart. The spectrogram restores a time axis at the cost of a fundamental trade-off between time and frequency resolution, controlled by the STFT window length;  $n_{fft} = 2048$  was a reasonable middle ground for this dataset. Keeping only the local-maxima peaks compresses a spectrogram by roughly three orders of magnitude while keeping the points most likely to survive noise, and pairing those peaks into  $(f_1, f_2, \Delta t)$  hashes

makes the resulting fingerprint vastly more selective than single frequencies on their own — the entire reason the offset histogram for a correct match shows one sharp, isolated spike rather than a noisy spread.

The system tolerated white noise down to at least  $-10$  dB SNR in our tests, and tolerated time stretching up to  $\pm 10\%$ , but a pitch shift of just one semitone was enough to destroy almost the entire match, even though the shifted clip sounds practically identical to a human ear. The cause is structural: the hash keys are tied to the *exact* integer frequency bins and frame counts produced by the STFT, so any transformation that moves those integers — even one a person can barely perceive — breaks an exact-match lookup table. Replacing the absolute-frequency hash with a pitch-ratio hash ( $\text{round}(f_2/f_1, 2), \Delta t$ ) fixed exactly this failure mode in our tests, recovering correct identification at every pitch shift tried, at the cost of a coarser, slightly less selective key.

These same fingerprinting and matching functions are reused unchanged in the Q3B Streamlit application.

## Appendix: Song library used

A Day In The Life, A Hard Day's Night, Across The Universe, Back In The U.S.S.R., Blackbird, Bohemian Rhapsody, Can't Buy Me Love, Crazy Little Thing Called Love, Day Tripper, Don't Stop Me Now, Drive My Car, Eight Days A Week, Eleanor Rigby, Get Back, Hello, Goodbye, Help!, Helter Skelter, Hey Jude, I Am The Walrus, I Saw Her Standing There, I Want It All, I Want To Hold Your Hand, I'll Follow The Sun, I've Got A Feeling, In My Life, Killer Queen, Let It Be, Love Me Do, Lucy In The Sky With Diamonds, Never Gonna Give You Up, Norwegian Wood (This Bird Has Flown), Penny Lane, Radio Ga Ga, Revolution, Sgt. Pepper's Lonely Hearts Club Band, She Said She Said, Somebody To Love, Something, Taxman, The Long And Winding Road, Two Of Us, Under Pressure, We Are The Champions, We Can Work It Out, We Will Rock You, While My Guitar Gently Weeps, With A Little Help From My Friends, Within You Without You, Yesterday, You Really Got A Hold On Me.